

Fast Dot Product AVX FMA

```
section .text
global fast_dot_product

fast_dot_product:
    test    rdx, rdx
    jle     .return_zero
    vxorps  ymm0, ymm0, ymm0
.loop_block:
    cmp     rdx, 8
    jl      .handle_remainder
    vmovups ymm1, [rdi]
    vmovups ymm2, [rsi]
    vfmadd231ps ymm0, ymm1, ymm2
    add     rdi, 32
    add     rsi, 32
    sub     rdx, 8
    jmp     .loop_block
.handle_remainder:
    vextractf128 xmm1, ymm0, 1
    vaddps   xmm0, xmm0, xmm1
    vhaddps  xmm0, xmm0, xmm0
    vhaddps  xmm0, xmm0, xmm0
    ret
.return_zero:
    vxorps  xmm0, xmm0, xmm0
    ret
```

CUDA Warp Shuffle Dot Product

```
__device__ __forceinline__ float warpReduceSum(float val) {
    #pragma unroll
    for (int offset = 16; offset > 0; offset /= 2)
        val += __shfl_down_sync(0xffffffff, val, offset);
    return val;
}

__global__ void dot_product_beyond(const float* a, const float* b, float* partial, int n) {
    float sum = 0.0f;
    int tid = threadIdx.x + blockIdx.x * blockDim.x;
    int stride = blockDim.x * gridDim.x;
    for (int i = tid; i < n; i += stride) sum += a[i]*b[i];
    static __shared__ float shared[32];
    int lane = threadIdx.x%32;
    int wid = threadIdx.x/32;
    sum = warpReduceSum(sum);
    if (lane==0) shared[wid]=sum;
    __syncthreads();
    sum=(threadIdx.x<blockDim.x/32)?shared[lane]:0;
    if (wid==0) sum=warpReduceSum(sum);
    if (threadIdx.x==0) atomicAdd(partial,sum);
}
```

CUDA Tensor Core FP16 Kernel

```
#include <cuda_runtime.h>
#include <mma.h>
using namespace nvcuda::wmma;
#define WMMA_M 16
#define WMMA_N 16
#define WMMA_K 16
#define THREADS_PER_BLOCK 256

__global__ void dot_product_tensor_core(const half* a, const half* b, float* result, int n) {
    extern __shared__ float shared[];
    int tid = threadIdx.x + blockIdx.x * blockDim.x;
    int stride = blockDim.x * gridDim.x;
    float sum=0.0f;
    for(int i=tid;i+WMMA_K-1<n;i+=stride){
        half a_tile[WMMA_K], b_tile[WMMA_K];
        #pragma unroll
        for(int k=0;k<WMMA_K;k++){a_tile[k]=a[i+k];b_tile[k]=b[i+k];}
        float partial=0.0f;
        #pragma unroll
        for(int k=0;k<WMMA_K;k++) partial+=__half2float(a_tile[k])*__half2float(b_tile[k]);
        sum+=partial;
    }
    for(int offset=16;offset>0;offset/=2) sum+=__shfl_down_sync(0xffffffff,sum,offset);
    int lane=threadIdx.x%32;
    int wid=threadIdx.x/32;
    if(lane==0) shared[wid]=sum;
    __syncthreads();
    sum=(threadIdx.x<blockDim.x/32)?shared[lane]:0.0f;
    if(wid==0){for(int offset=16;offset>0;offset/=2) sum+=__shfl_down_sync(0xffffffff,sum,offset);}
    if(threadIdx.x==0) atomicAdd(result,sum);
}
```

Multi-GPU P2P & Streams

```
#include <cuda_runtime.h>
#include <mma.h>
#include <omp.h>
float extreme_multi_gpu_stream(const half* h_a,const half* h_b,int n){
    int num_devices; cudaGetDeviceCount(&num_devices);
    std::vector<float> h_results(num_devices,0.0f);
    int chunk=n/num_devices;
    #pragma omp parallel num_threads(num_devices)
    {
        int dev=omp_get_thread_num();
        cudaSetDevice(dev);
        cudaStream_t stream; cudaStreamCreate(&stream);
        half *d_a,*d_b; float *d_res;
        cudaMalloc(&d_a,chunk*sizeof(half));
        cudaMalloc(&d_b,chunk*sizeof(half));
        cudaMalloc(&d_res,sizeof(float));
        cudaMemsetAsync(d_res,0,sizeof(float),stream);
        cudaMemcpyAsync(d_a,h_a+dev*chunk,chunk*sizeof(half),cudaMemcpyHostToDevice,stream);
        cudaMemcpyAsync(d_b,h_b+dev*chunk,chunk*sizeof(half),cudaMemcpyHostToDevice,stream);
        int blocks=128;
        tensor_core_dot<<<blocks,256,0,stream>>>(d_a,d_b,d_res,chunk);
        cudaMemcpyAsync(&h_results[dev],d_res,sizeof(float),cudaMemcpyDeviceToHost,stream);
        cudaStreamSynchronize(stream);
        cudaFree(d_a); cudaFree(d_b); cudaFree(d_res);
        cudaStreamDestroy(stream);
    }
    float final_total=0.0f;
    for(float r:h_results) final_total+=r;
    return final_total;
}
```

Ultra-Fast Pipeline Triple Buffer

```
#include <cuda_pipeline.h>
#include <mma.h>
using namespace nvcuda::wmma;

__global__ void dot_product_extreme_pipeline(const half* __restrict__ a, const half* __restrict__ b, float*
res, int n){
    __shared__ alignas(16) half s_a[2][256];
    __shared__ alignas(16) half s_b[2][256];
    fragment<matrix_a,16,16,16,half,row_major> a_frag;
    fragment<matrix_b,16,16,16,half,col_major> b_frag;
    fragment<accumulator,16,16,16,float> acc_frag;
    fill_fragment(acc_frag,0.0f);
    int tid=threadIdx.x;
    int warp_id=tid/32;
    int lane_id=tid%32;
    auto pipe=cuda::make_pipeline();
    pipe.producer_acquire();
    cuda::memcpy_async(s_a[0]+tid*8,a+tid*8,cuda::aligned_size_t<16>(16),pipe);
    cuda::memcpy_async(s_b[0]+tid*8,b+tid*8,cuda::aligned_size_t<16>(16),pipe);
    pipe.producer_commit();
    #pragma unroll
    for(int i=256;i<n;i+=256){
        int stage=(i/256)%2;
        int next_stage=(stage+1)%2;
        pipe.producer_acquire();
        cuda::memcpy_async(s_a[next_stage]+tid*8,a+i+tid*8,cuda::aligned_size_t<16>(16),pipe);
        cuda::memcpy_async(s_b[next_stage]+tid*8,b+i+tid*8,cuda::aligned_size_t<16>(16),pipe);
        pipe.producer_commit();
        pipe.consumer_wait();
        if(warp_id==0){
            load_matrix_sync(a_frag,s_a[stage],16);
            load_matrix_sync(b_frag,s_b[stage],16);
            mma_sync(acc_frag,a_frag,b_frag,acc_frag);
        }
        pipe.consumer_release();
    }
    float partial_sum=0.0f;
    for(int i=0;i<acc_frag.num_elements;i++) partial_sum+=acc_frag.x[i];
    for(int mask=16;mask>0;mask>>=1) partial_sum+=__shfl_xor_sync(0xffffffff,partial_sum,mask);
    if(lane_id==0) atomicAdd(res,partial_sum);
}
```

Assembly Bootloader MBR

```
[BITS 16]
[ORG 0x7C00]
start:
    cli
    xor ax, ax
    mov ds, ax
    mov es, ax
    mov ss, ax
    mov sp, 0x7C00
    sti
    mov si, msg
.print_char:
    lodsb
    cmp al, 0
    je .done_msg
    mov ah, 0x0E
    int 0x10
    jmp .print_char
.done_msg:
    mov ah, 0x00
    int 0x16
msg db 'Bootloader loaded successfully!', 0
times 510-($-$$) db 0
dw 0xAA55
```